

Date _____

Day _____

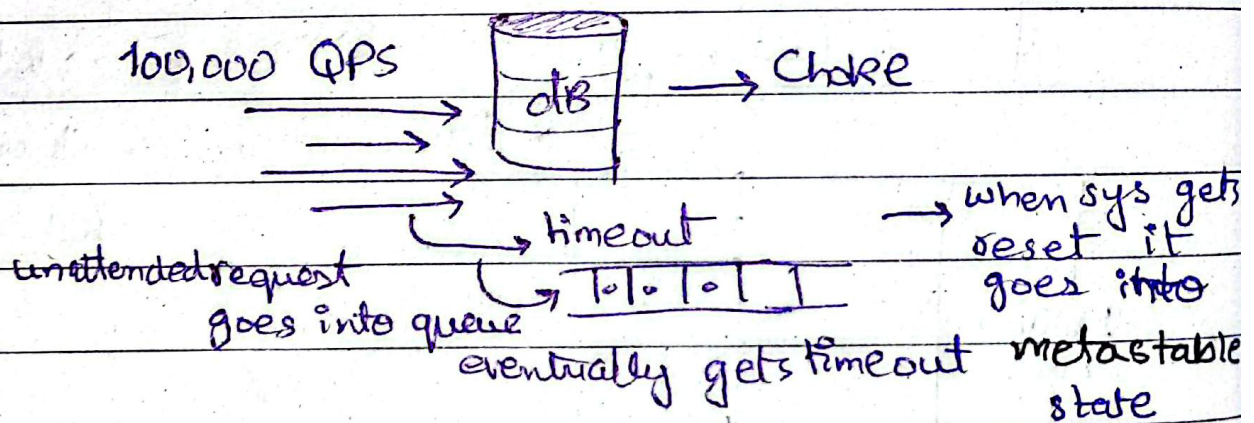
② Limited SSD / HDD

↳ largest hard disk upto date

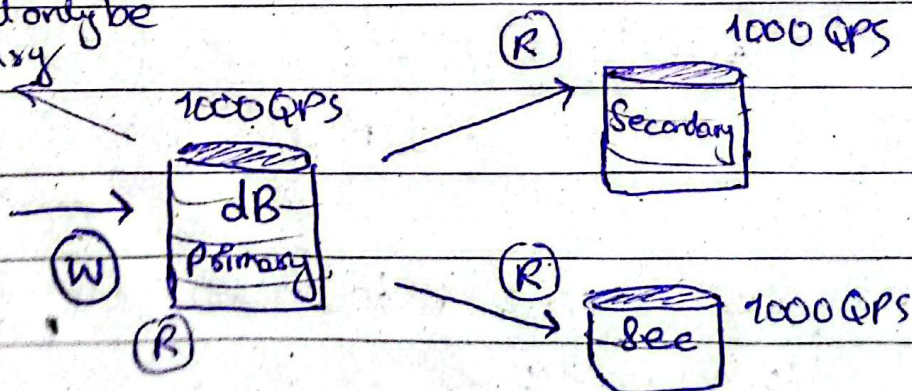
LCA → 250TB (25,000 - 30,000 USD)

③ Through put

↳ One db not enough because of its limited capacity, compute, I/O, threads.



write will only be on Primary



overall throughput
3000 QPS

↓
But only on
READ

↳ WRITE still on 1000 QPS

Page No. _____

Date _____

Day _____

• Approaches To Problems

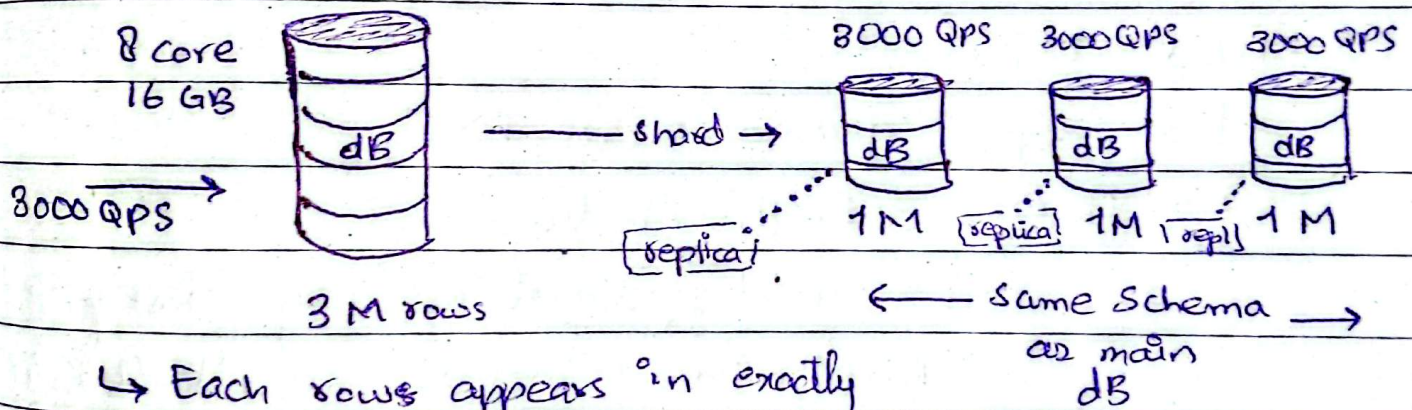
→ Horizontal Scaling

① Approach #01 : Sharding / Partitioning

↳ Sharding

↳ Horizontal Scaling of database

↳ Data is split across multiple dB instances



↳ Each row appears in exactly one shard

• Types Of Sharding

① Range-based Sharding

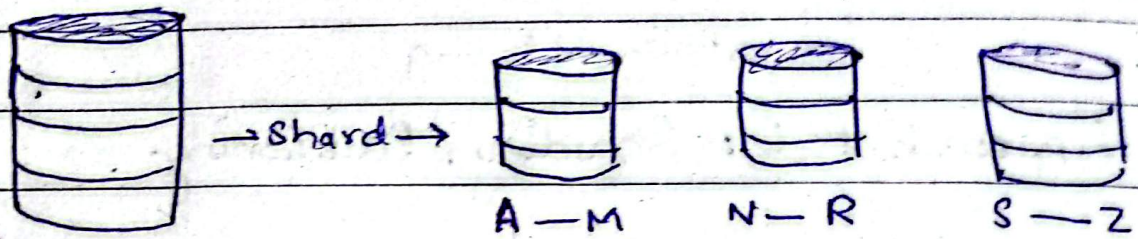
↳ Also called Horizontal Sharding.

↳ We divide data based on some range.

↳ For ex, divide data based on student_name

Date _____

Day _____



↳ Query performance ↑, becuz
of parallelization

Backed then decide on which
db to store the data

Trade-offs

- ↳ Complex querying across the shards
- ↳ Uneven data distribution

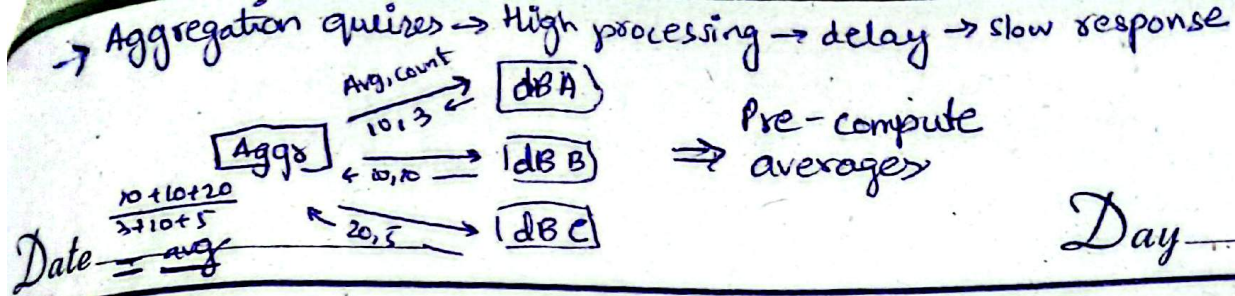
↳ it might happen that we only get reqs for
A-M student names. So all 9000 QPS
will hit single db.

↳ That db becomes HOTSPOT

② Key-based Sharding

↳ Also known as Hash based sharding.

↳ Based on an ID value we hash them and
we distribute data according to that



$$\frac{\text{hashValue}}{\text{ServerIndex}} = \text{hashfunc}(\text{key}) = \text{key} \% \text{no. of servers}$$

student_id	Shard No.	Random Traffic
0	$7.3 = 0$	↓
1	$7.3 = 1$	Distributes Randomly (But only on WRITE)
2	$7.3 = 2$	
3	$7.3 = 0$	
4	$7.3 = 1$	

→ Every Read/Write request for student_id = 3 will go to Shard # 0

• Trade - Offs

↳ When increasing the no-of shards to increase throughput — for say $n=10$

↳ We now have to → Rewrite

↳ Redistribute

↳ Merge all data in single dB and then redistribute it. VERY Painful!

To

↳ Reduce the data migration

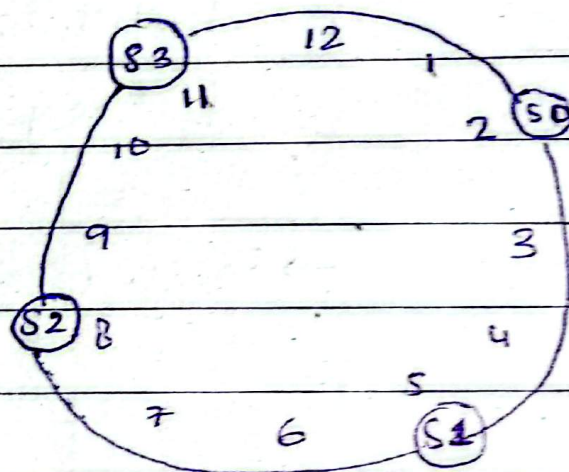
↳ We do consistent hashing

Date _____

Day _____

• Consistent Hashing

↳ Hash function is used to map the servers based on their IP Addresses, onto a ring



↳ We have predefined slots on the ring the servers are placed in one of those based on the hash value

↳ Now instead of mapping the data directly on the servers we map it on the ring just like we mapped the servers.

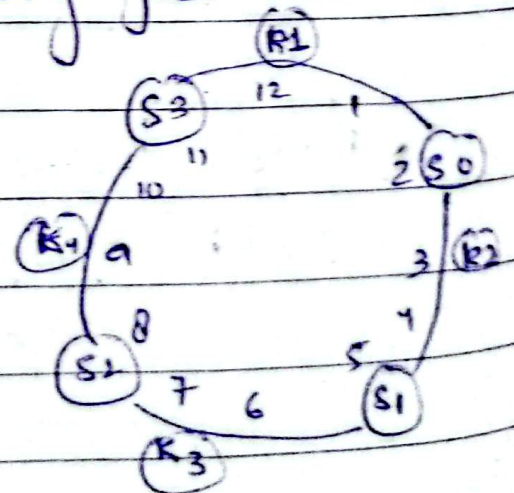
↳ For eg: $\text{hash}(\text{Key 1}) = 12$

$\text{hash}(\text{Key 2}) = 3$

$\text{hash}(\text{Key 3}) = 7$

$\text{hash}(\text{Key 4}) = 9$

key = data



Placing key on ring

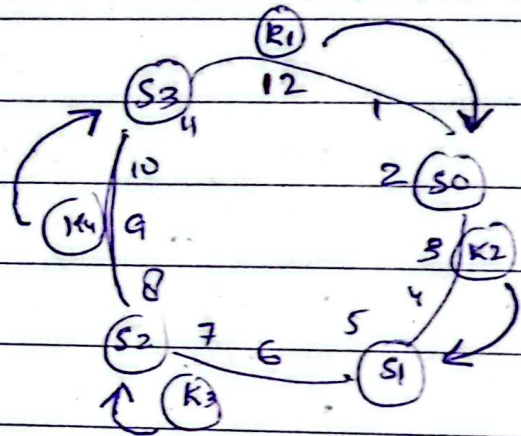
- Consistent Hashing widely used in
 - ↳ Amazon Dynamo Db
 - ↳ Apache Cassandra ⇒ system optimized for WRITES

Date _____

Day _____

↳ Now ~~we~~ we move the keys clock-wise and it is assigned to the first server it encounters

↳ What Happens when a Server Removed Or Add?



For Ex: S1 gets removed.

↳ So now only the keys in S1 needs to be migrated to S2. Other than that no migrations needed. ↳ Only keys b/w 3rd & 7th needs to be remapped

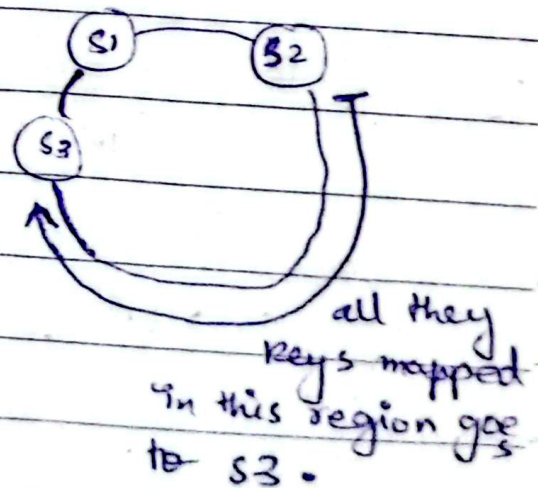
For Ex: S4 added b/w S3 & S0 at 1st pos.

↳ Now only the keys b/w S3 & S4 needs to be migrated.
 or 11th or 1st pos

• Problem:

↳ If the hash functions maps the server closely than one server becomes hotspot

Fix: We make virtual servers in the ring.



Page No. _____

→ JOINS adds latency in the system

Date _____ SQL not optimized for this

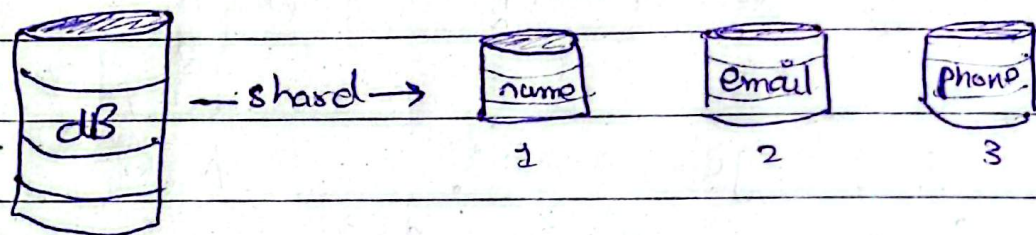
Day _____

③ Vertical Sharding

↳ Distribute data based on columns

↳ We have 3 columns, first_name, email, phone_no

↳ Now each shard will have one column.



↳

↳ Aggregation queries are fast

↳ Problems:

↳ Certain shards become HOTSPOT

↳ Changes in schema becomes difficult

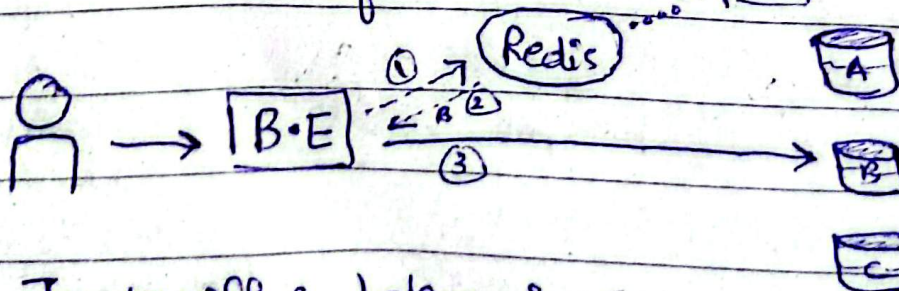
↳ If we want to retrieve single row → cross shard query

↳ Not optimized for Row-by-row queries

④ Directory-Based Sharding

↳ Make a lookup-service (Redis).

↳ The backend request that service to get the shard # of the data



Trade-off: Latency increase, Redis needs replica

Page No. _____

Date _____

Day _____

② Approach #02 : Vertical Scaling

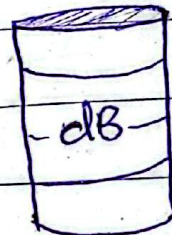
→ Increasing the compute of the existing machine

Clouds:



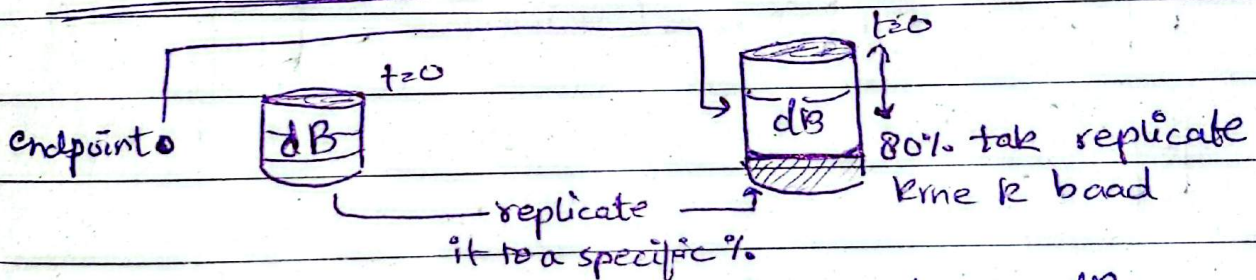
VCPU: 2 cores
RAM: 2 GB

... downtime ...
very little
in cloud.



VCPU: 8 cores
RAM: 32 GB

In-house Data Center:



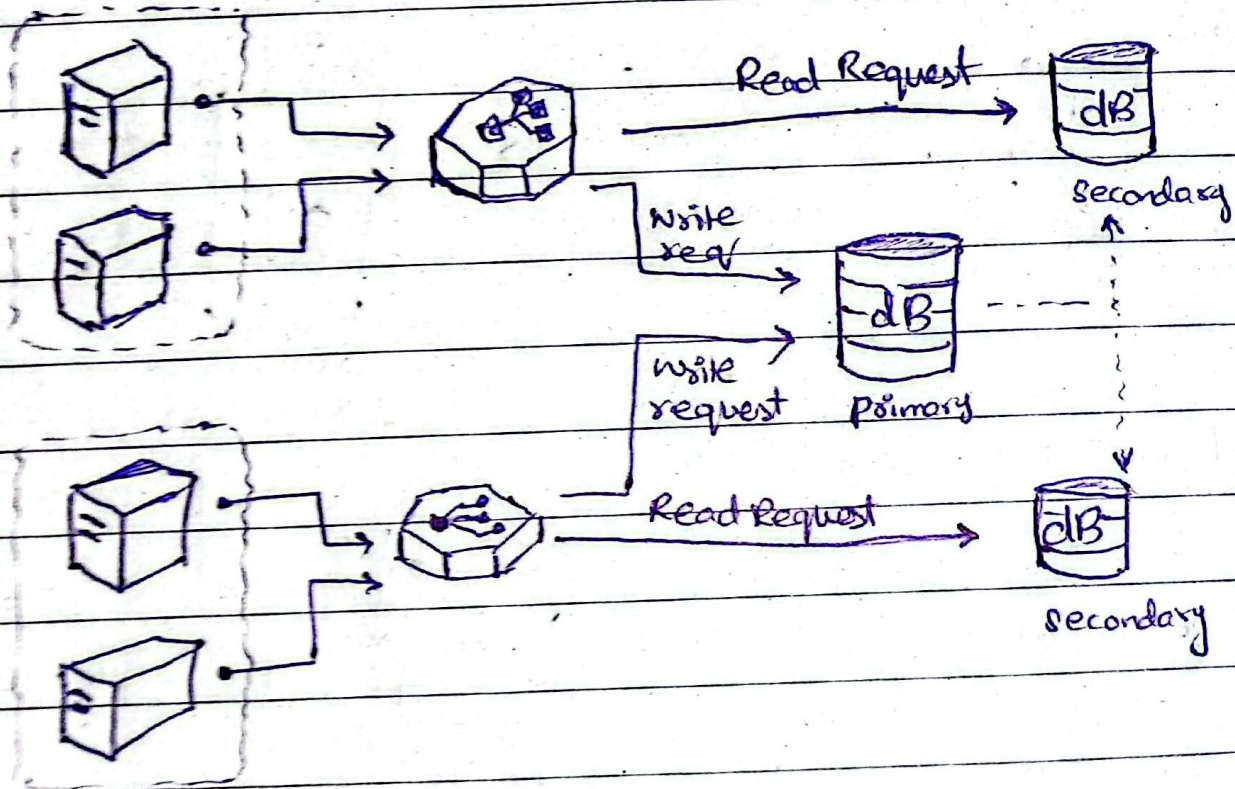
and after that route the req to new dB
→ through this we can reduce downtime

SLA → we have defined 99% ⁹⁹ ~~downtime~~ availability
so can't shutdown dB for 10 days

Date _____

Day _____

(3) Approach # 03 : Primary/Secondary / Master/Slave / Leader/Follower



Problems:

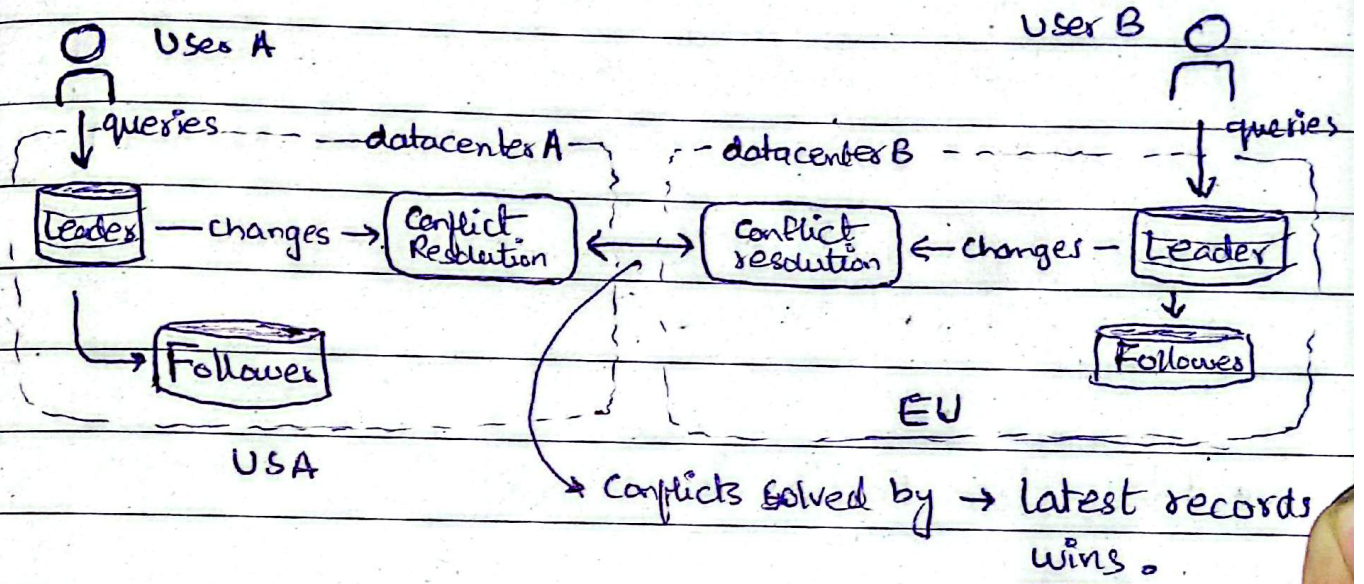
↳ Eventual Consistency

↳ Bad for Fintech Companies

Date _____

Day _____

• Multi Leader Replication



↳ Same DB in two regions

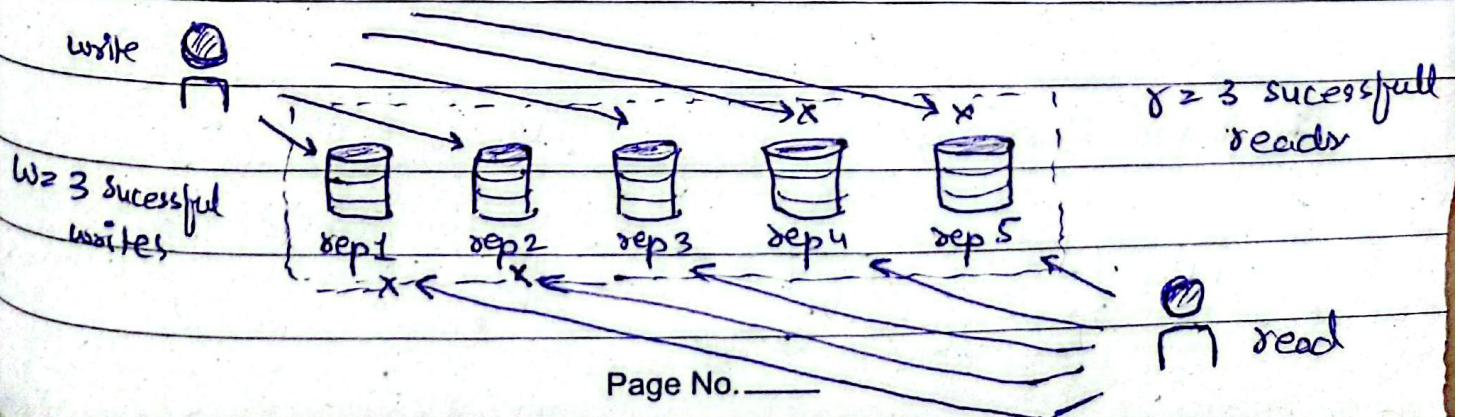
↳ to reduce latency, two leaders are deployed in multiple regions.

↳ when both leaders try to sync the data, the latest record wins

↳ Chance of data loss increases

• Leaderless

$n = 5$ replicas



Page No. _____

Date _____

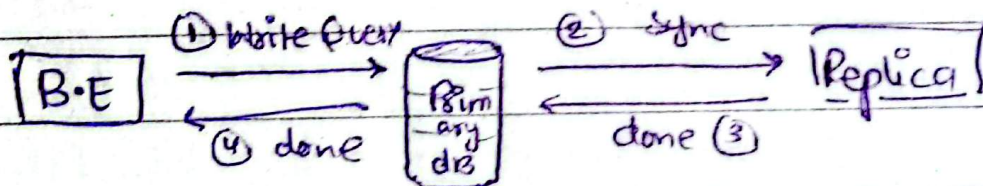
Day _____

↳ We get 3 records from 3 successful reads
if all of them have same it means data is consistent

→ AWS DynamoDB } uses readers
→ Cassandra }

• How does Replication works?

① Synchronous Replicas



↳ Replica always in Sync

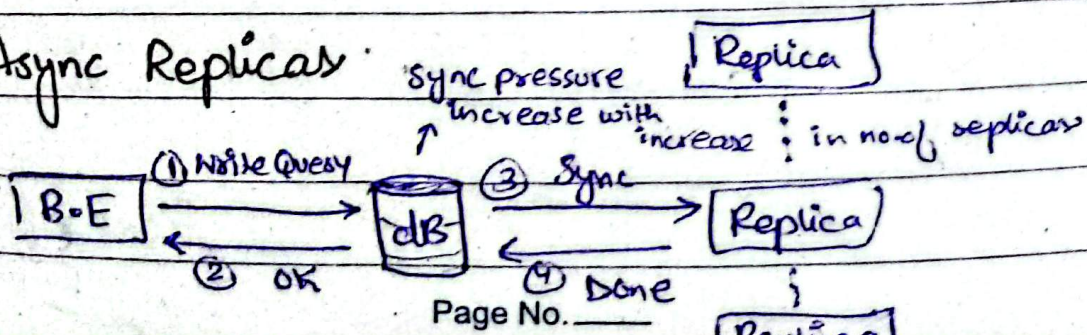
Cons:

→ replica may be busy → reqs wait

↳ Increases latency

↳ Sync Fail → transaction reverts

② Async Replicas



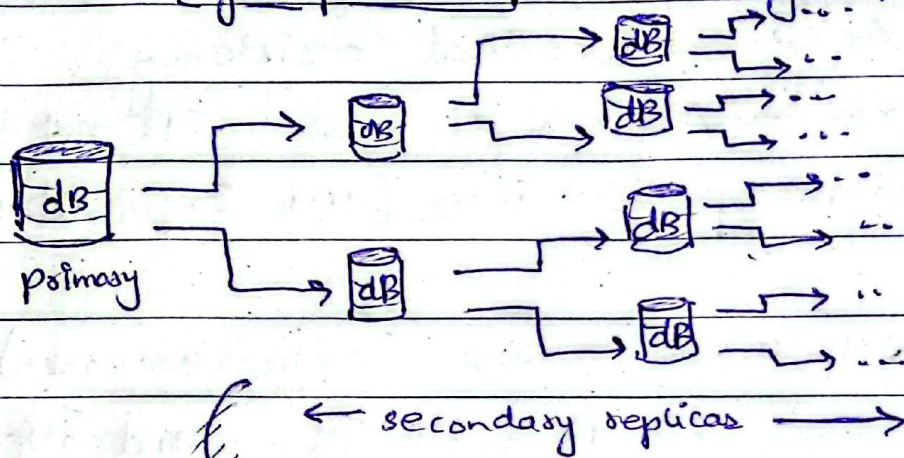
Page No. _____

Date _____

Followed by OpenAI & Azure

Day _____

↳ to reduce Sync pressure ⇒ Cascading Replica



PROs:

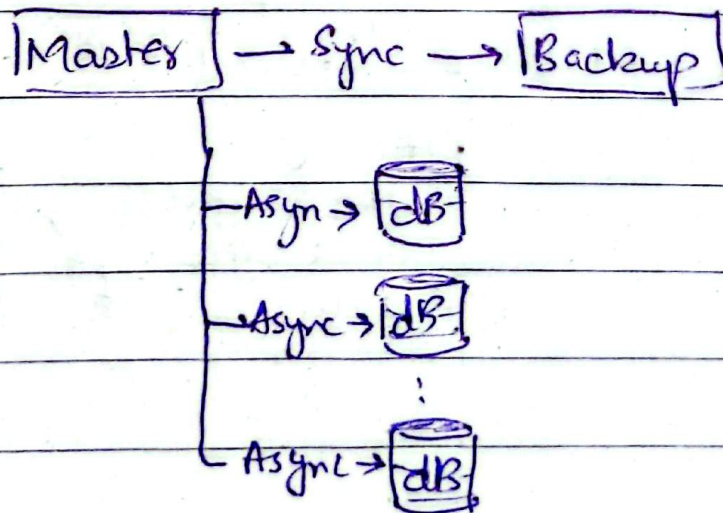
↳ Latency ↓

↳ Highly Available

Cons:

↳ eventual consistency

Hybrid: (For Disaster Management) → In disaster atleast we have one backup



Page No. _____

Date _____

Day _____

Q: User updates his profile (WRITE) → DB is updated
Async → eventual consistency.

↳ Due to Async replica user will not be able to view updated info (User Exp will be bad)

↳ In these cases (critical usecases) after write on Primary DB, and after a window 1-2 min we read for the primary one instead of Replica.

↳ Read After Write

↳ Read After deletion

⑧ Approach #05 : Indexing & Optimization

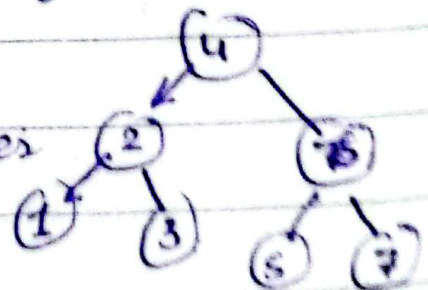
Q: How are indexes store?

A Binary search Tree with more than 2 childrens
↳ A single node can hold multiple keys

↳ B-Tree (Balance Tree)

↳ id → 1, 2, ..., 7

All leaf nodes are at same depth

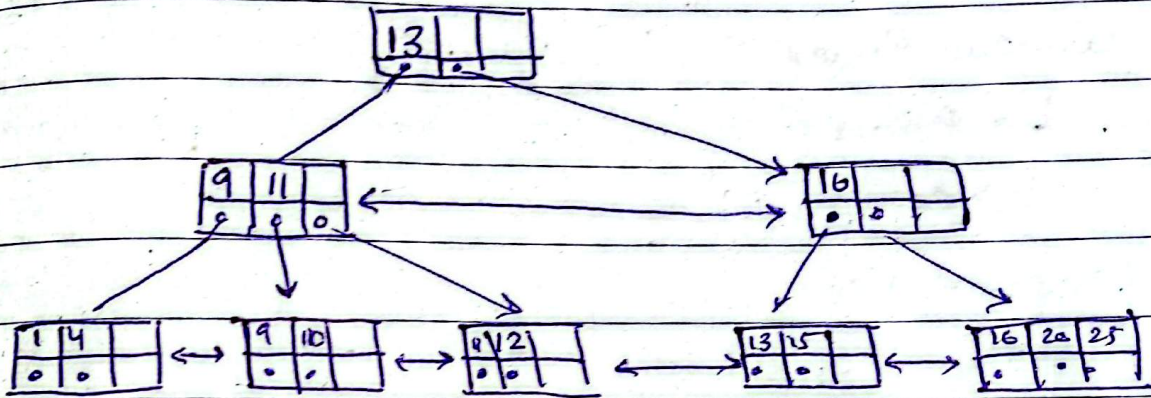


→ In a composite key index, the order of attributes should be same as order of index.
index → (city, gender) → Query SELECT * WHERE city = " " AND gen = " "

Date _____

Day _____

• Optimization of B-Trees → B⁺ Trees



- ↳ Can store multiple key in a single node
- ↳ Siblings can talk to each other
- ↳ Optimized for reads

Q: Why INSERTS are slow?

- ↳ Because every time a new record is made we need to balance the tree.
- ↳ Therefore multiindexing is slow
- ↳ Split & Merge algo is performed on WRITE

→ If a clustered key is changed than "Reshuffling" processes is performed.

Date _____

Day _____

→ Types Of Indexes

- B⁺ Trees (Default)

- Unique Index

→ Email

→ CNIC

- Composite Key

| city = Karachi

| gender = Male

- Hash Index → direct mapping

id = 10 : 8 of records

= 12 : 9

- Full Text Search

↳ Applied on columns having long text

↳ Usefull in search engines

- GIN / GIST → R-Trees

↳ JSON / Arrays

↳ long / lat